Strukturalni paterni u OCULIS sistem

1. Uvod

OCULIS je web sistem namijenjen radu optike. Omogućava korisnicima pregled i kupovinu proizvoda, upravljanje korpom, kreiranje narudžbi, obradu plaćanja, zakazivanje termina pregleda vida, vođenje elektronskog kartona i slanje obavijesti korisnicima.

Pošto sistem sadrži više međusobno povezanih klasa, važno je da odnosi između njih budu jasno organizovani. To se posebno vidi kod obrade narudžbe, gdje zajedno učestvuju korpa, narudžba, plaćanje, obračun popusta i servis za obavijesti.

U klasni dijagram OCULIS sistema dodana su dva odabrana paterna: Facade i Adapter. Cilj je da se postojeći model proširi na pregledan način, bez nepotrebnog opterećivanja postojećih klasa.

2. Potreba za strukturalnim paternima

Strukturalni paterni koriste se za organizaciju odnosa između klasa i objekata. Oni pomažu da složeniji dijelovi sistema budu povezani jasnije, uz manju zavisnost između klasa.

U OCULIS sistemu jedna funkcionalnost često uključuje više klasa. Na primjer, obrada narudžbe ne znači samo kreiranje narudžbe, nego i rad sa korpom, obračun popusta, izvršavanje plaćanja i slanje obavijesti korisniku. Kada bi kontroler direktno komunicirao sa svim tim klasama, sistem bi bio teži za čitanje, testiranje i održavanje.

Također, sistem može koristiti vanjske servise, kao što je PayPal servis za plaćanje. Takav servis obično ima drugačiji interfejs od onoga koji koristi aplikacija, pa je potrebno omogućiti prilagođavanje bez mijenjanja postojećih klasa.

3. Pregled strukturalnih paterna

U strukturalne paterne spadaju Adapter, Bridge, Composite, Decorator, Facade, Proxy . U nastavku je dat kratak pregled njihove moguće primjene u OCULIS sistemu.

- Adapter omogućava povezivanje klasa ili servisa koji imaju nekompatibilne interfejse. U OCULIS sistemu pogodan je za povezivanje sa vanjskim servisom za plaćanje.
- Bridge razdvaja apstrakciju od implementacije. Mogao bi se koristiti kod obavijesti, gdje postoje različite vrste obavijesti i različiti kanali slanja.
- Composite omogućava da se pojedinačni objekti i grupe objekata tretiraju na isti način. U ovom sistemu mogao bi se koristiti kod hijerarhijske organizacije kategorija proizvoda.
- Decorator omogućava dodavanje novih funkcionalnosti objektu bez mijenjanja njegove osnovne klase. Primjer su dodatne opcije proizvoda, poput zaštitnog sloja, dodatne garancije ili poklon pakovanja.
- Facade pruža jednostavan interfejs prema složenom dijelu sistema. U OCULIS sistemu najprirodnije odgovara procesu obrade narudžbe.
- Proxy kontroliše pristup određenom objektu. Mogao bi se koristiti za zaštitu pristupa elektronskom kartonu korisnika.

4. Odabrani paterni za OCULIS sistem

Za novi klasni dijagram odabrana su dva strukturalna paterna:

- Facade
- Adapter

Ovi paterni su izabrani jer se direktno uklapaju u postojeće potrebe sistema. Facade pojednostavljuje obradu narudžbe, dok Adapter omogućava povezivanje sa vanjskim servisom za online plaćanje.

5. Facade patern

Facade patern je dodat kroz klasu NarudzbaFacade. Ova klasa predstavlja jedinstvenu ulaznu tačku za proces obrade narudžbe.

Obrada narudžbe obuhvata više koraka: dobavljanje ukupne cijene iz korpe, obračun popusta, izvršavanje plaćanja, ažuriranje narudžbe i slanje obavijesti korisniku. Bez ovog paterna, NarudzbaController bi morao direktno pozivati više različitih klasa.

Uvođenjem klase NarudzbaFacade, kontroler poziva jednu centralnu metodu:

```
+ obradiNarudzbu(narudzba : Narudzba) : void
```

Klasa NarudzbaFacade povezana je sa sljedećim klasama:

- Narudzba
- Korpa
- Placanje
- ObracunPopusta
- ObavijestService

Na ovaj način se smanjuje složenost kontrolera, a komunikacija između klasa postaje preglednija i lakša za praćenje.

6. Adapter patern

Adapter patern je dodat kroz klase PayPalAdapter i ExternalPayPalService. U postojećem sistemu postoji interfejs IPlacanje, koji definiše metodu:

```
+ izvrsiPlacanje(iznos : double) : StatusPlacanja
```

Postojeće klase KarticnoPlacanje i GotovinskoPlacanje implementiraju ovaj interfejs. Međutim, vanjski PayPal servis može imati drugačiji naziv metode i drugačiji povratni tip, na primjer:

```
+ makePayment(amount : double) : bool
```

Zbog toga se uvodi klasa PayPalAdapter. Ona implementira IPlacanje i prevodi poziv iz OCULIS sistema u poziv koji razumije vanjski PayPal servis.

Na klasnom dijagramu prikazano je sljedeće:

- PayPalAdapter implementira IPlacanje.
- PayPalAdapter koristi ExternalPayPalService.
- ExternalPayPalService predstavlja vanjski servis za PayPal plaćanje.

Ovim rješenjem ostatak sistema i dalje koristi isti interfejs za plaćanje, bez potrebe da poznaje detalje rada PayPal servisa.

7. Veza između odabranih paterna

Facade i Adapter se povezuju kroz proces obrade narudžbe. NarudzbaFacade koordinira obradu narudžbe i u tom procesu koristi plaćanje preko interfejsa IPlacanje. Jedna od konkretnih implementacija tog interfejsa je PayPalAdapter.

Tako Facade pojednostavljuje tok obrade narudžbe, dok Adapter omogućava da se vanjski servis uklopi u postojeći način rada aplikacije.

8. Zaključak

Dodavanjem strukturalnih paterna OCULIS sistem postaje pregledniji i lakši za održavanje. Facade patern pojednostavljuje proces obrade narudžbe tako što uvodi klasu NarudzbaFacade, koja objedinjuje rad sa korpom, narudžbom, plaćanjem, obračunom popusta i obavijestima.

Adapter patern omogućava povezivanje sistema sa vanjskim PayPal servisom bez promjene postojećeg interfejsa za plaćanje. Klasa PayPalAdapter prilagođava vanjski servis interfejsu IPlacanje.

Na taj način sistem se može proširivati bez narušavanja postojećeg modela klasa, a postojeće klase ostaju jasnije, jednostavnije i spremnije za buduće izmjene.